.NET MCP (dotnet-mcp)

The **.NET MCP Server** connects an AI assistant to the .NET SDK as structured tools and resources. In this project it is the **structured CLI** and the way the final build gate is run; SageFs (`docs/sagefs.typ`) remains the F# **inner loop**.

What is installed

Piece	Value
Package	`Community.Mcp.DotNet` (Jon Galloway)
MCP registry identity	`io.github.jongalloway/dotnet-mcp`
Version server reports	`1.1.2`
Transport	stdio
Launch	`dnx Community.Mcp.DotNet@1.* --yes`
Requires	.NET 10 SDK (provides `dnx`)
OpenCode V2 config	`~/.config/opencode/opencode.json` ⟶ `mcp.servers.dotnet`
OpenCode V1 config	`~/.config/opencode/opencode-v1.json` ⟶ `mcp.dotnet`
Skill	`~/.config/opencode/skills/dotnet-mcp/SKILL.md`

The name is load-bearing and ambiguous. There is also an *unlisted, deprecated* NuGet package literally called `dotnet-mcp` (Devlooped). The server this repo uses is `Community.Mcp.DotNet`. Do not install the other one.

Install and verify

```
opencode mcp add dotnet -- dnx Community.Mcp.DotNet@1.* --yes
opencode mcp list        # ✓ dotnet  connected
```

The config sets `DOTNET_ROOT` and prepends `/usr/share/dotnet` and `~/.dotnet/tools` to `PATH`, because `dnx` needs `dotnet` on PATH exactly as SageFs does. Under Code Mode the tools are `tools.dotnet.<tool>(...)`; otherwise they appear as `dotnet_<tool>`.

The 12 tools

Every functional tool is consolidated: one tool, many operations, chosen by an `action` parameter.

Tool	Actions
`dotnet_project`	New, Restore, Build, Run, Test, Publish, Clean, Analyze, Dependencies, Validate, Pack, Watch, Format, Stop, Logs, ListTemplateOptions, SetProperty, GetProperty, RemoveProperty, AddItem, RemoveItem, ListItems
`dotnet_solution`	Create, Add, List, Remove, Analyze, Dependencies, Validate
`dotnet_package`	Add, Remove, Search, Update, List, AddReference, RemoveReference, ListReferences, ClearCache
`dotnet_sdk`	Version, Info, ListSdks, ListRuntimes, ListTemplates, SearchTemplates, TemplateInfo, ListTemplatePacks, InstallTemplatePack, UninstallTemplatePack, FrameworkInfo, CacheMetrics, ConfigureGlobalJson
`dotnet_tool`	Install, List, Update, Uninstall, Restore, CreateManifest, Search, Run
`dotnet_workload`	List, Info, Search, Install, Update, Uninstall

Tool	Actions
`dotnet_ef`	MigrationsAdd, MigrationsList, MigrationsRemove, MigrationsScript, DatabaseUpdate, DatabaseDrop, DbContextList, DbContextInfo, DbContextScaffold
`dotnet_dev_certs`	CertificateTrust, CertificateCheck, CertificateClean, CertificateExport, SecretsInit, SecretsSet, SecretsList, SecretsRemove, SecretsClear
`dotnet_help`	— (takes a `command`)
`dotnet_server_info`	—
`dotnet_server_capabilities`	—
`dotnet_server_metrics`	Get, Reset

Resources

`dotnet://sdk-info`, `dotnet://runtime-info`, `dotnet://templates`, `dotnet://frameworks`, `dotnet://telemetry-data`.

(The upstream README also lists `dotnet://workspace`; the running v1.1.2 server does **not** register it, and reading it fails with *"Unknown resource URI"*.)

Read them without running a command:

```
await tools.opencode.list_mcp_resources({ server: "dotnet" });
await tools.opencode.read_mcp_resource({ server: "dotnet", uri: "dotnet://sdk-
info" });
```

Using it on this repo

Alpha phase. The MCP build and test actions invoke MSBuild. Do not use them for development during alpha; use SageFs for compile, run, and test. `just check` and the AOT recipes are release-time only.

```
// Build the project.
await tools.dotnet.dotnet_project({
  action: "Build",
  project: "/home/shalok/Projects/Scale/bases/scale-cli/scale.fsproj",
  configuration: "Debug",
});

// The example acceptance suite (1530 checks) lives in the JIT test
// deployable; the product base has no test modes (D176). `just check` also runs
// the granular/Verify suite (`appArgs: "--granular-tests"`, JIT only, D23).
// NOTE `appArgs` is a STRING, not an array.
await tools.dotnet.dotnet_project({
  action: "Run",
  project: "/home/shalok/Projects/Scale/projects/scale-tests/scale-tests.fsproj",
  appArgs: "--tests",
});

// Structured inspection (these take `projectPath`, not `project`).
await tools.dotnet.dotnet_project({ action: "Validate", projectPath: "bases/scale-
cli/scale.fsproj" });
await tools.dotnet.dotnet_project({ action: "Dependencies", projectPath: "bases/
scale-cli/scale.fsproj" });
```

`Validate` returns for example `{"success":true,"isValid":true,"errors":[],"warnings":[],"recommendations":[...]}`.

Three corrections the upstream README does not make

These were found by using the server, and they matter.

1. `Test` is not this repo's gate

Scale's suite is a example `Exe` (`Tests.runAll ()`), not a discovered test framework. `dotnet test` therefore finds nothing and **exits 0**:

```
dotnet test bases/scale-cli/scale.fsproj
  Determining projects to restore...
  All projects are up-to-date for restore.
Exit Code: 0        ← green, and it ran zero tests
```

`--list-tests` also returns nothing. This is the exact failure mode to avoid: a green gate that checked nothing. The gate is `just check` (both suites); the example suite is `Run` on `projects/scale-tests` with `appArgs: "--tests"`, which ends with `>>> ALL TESTS PASSED (1530) <<<`, and the granular/Verify suite is `appArgs: "--granular-tests"` (JIT only, ends with `>>> ALL EXPECTO TESTS PASSED <<<`). (SageFs `list_tests`/`run_tests` do see the granular-wrapped cases — see `docs/sagefs.typ` — but the acceptance suite is still the Exe.)

2. `Format` does not support F#

`action: "Format"` shells to `dotnet format`, which answers:

```
Could not format 'bases/scale-cli/scale.fsproj'. Format currently supports only C# and Visual Basic projects.
```

F# formatting here is **Fantomas** (`dotnet fantomas check .` / `dotnet fantomas .`), the project's authority. Do not reach for the MCP `Format` action in an F# repo. Formatting is a rework-boundary commit, not a per-change reflex; see `docs/roadmap.typ` section 1, item 10.

3. `machineReadable` is documented but absent

The upstream README describes `machineReadable: true`; the v1.1.2 tool schema does not contain the parameter (verified against `inputSchema.properties`), so setting it is ignored. The inspection actions (`Validate`, `Analyze`, `Dependencies`) already return JSON regardless.

Which tool for which job

- **Build / run the gate** → `dotnet_project` `Build` / `Run`.
- **Structure of a project** → `dotnet_project` `Validate`, `Analyze`, `Dependencies` (these take `projectPath`).
- **Packages** → `dotnet_package` `Search` / `Add` / `List`.
- **Environment facts** → the `dotnet://…` resources, or `dotnet_sdk`.
- **Another language's tooling** → not this server. F# uses Fantomas and SageFs; this server is the SDK surface around them.

Relationship to SageFs

- **SageFs** — F# **inner loop**: warm REPL, millisecond evals. Prove a change before writing it to a file.
- **dotnet-mcp** — **structured CLI** and **final gate**: build, run the suite, inspect structure, query the environment.

The house rule is unchanged: prove in the REPL, then run the full build and both unfiltered suites once, at the end.

Gotchas

- **`dnx` needs .NET 10.** On an older SDK `dnx` does not exist; the upstream README's manual `dotnet run --project` config is the fallback.
- **First call is slower.** `dnx` resolves and downloads the package on first use; later calls reuse the cache.
- **`project` vs `projectPath`.** `Validate`/`Analyze`/`Dependencies` require `projectPath`; most others use `project`.
- **`appArgs` is a string**, not an array — `appArgs: "--tests"`, not `["--tests"]`.
- **No `global.json` in the repo.** It lives above the checkout at `~/global.json`, so tooling and continuous integration can resolve a different SDK than local runs do. Worth fixing when continuous integration is set up. The server can write one for you: `dotnet_sdk({ action: "ConfigureGlobalJson", sdkVersion: "10.0.401", rollForward: "latestPatch" })` (verified: it writes a valid `global.json`).